

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

C02: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Debugging or Optimization task

Code of Conduct:

I _G.Puneeth(192524221) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Puneeth

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock.

Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Answer: Q1

Exact deadlock condition

P1 acquires R1 and, after the 100 ms delay, tries to acquire R2. Simultaneously P2 acquires R2 and, after its own 100 ms delay, tries to acquire R1. Each process is holding one resource while waiting indefinitely for the resource held by the other. This is a circular wait built on top of hold-and-wait: P1 → waits for R2 (held by P2) → waits for R1 (held by P1). All four necessary conditions for deadlock are present — mutual exclusion (locks are exclusive), hold-and-wait (each keeps its first lock while requesting the second), no preemption (locks are not forcibly taken back), and circular wait (P1→R2→P2→R1→P1) — so the system deadlocks permanently once both processes cross the wait(100 ms) point.

Optimisation (process logic unchanged)

Break the circular-wait condition by imposing a global lock ordering (resource-ordering protocol) at the resource-manager level rather than editing the processes' internal logic: define a fixed acquisition order for all shared resources (e.g., always R1 before R2, using resource IDs), and have the lock manager enforce this order transparently for every lock() call. Under this rule P2's lock(R2) request is internally re-sequenced so it must wait for R1 first, so both processes always request R1 then R2, eliminating the cyclic dependency. An equivalent alternative is to replace blocking lock() with a try-lock-with-timeout / back-off scheme so that a process failing to obtain its second lock releases the first and retries — this removes the hold-and-wait condition without altering what each process actually computes in its critical section.

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.

You analyse the scheduler log and find frequent transitions:

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Answer: Q2

Root cause

Frequent Ready → Running → Waiting → Ready transitions indicate the ready mix is dominated by I/O-bound processes with very short CPU bursts. Each process runs briefly, immediately blocks for I/O, and the scheduler must repeatedly context-switch to find the next ready process. The CPU spends a disproportionate share of time performing scheduling/dispatch overhead and sitting idle while all runnable processes are simultaneously blocked on I/O, rather than executing useful work — this matches a low degree of multiprogramming combined with a scheduling policy (e.g., plain FCFS or an overly small time quantum in Round Robin) that does not distinguish CPU-bound from I/O-bound behaviour, producing convoy-like idling instead of overlap between CPU and I/O activity.

System-level optimisation

Increase the degree of multiprogramming (admit more ready processes into memory) so that while one process is blocked on I/O, others are available to use the CPU, and replace the flat scheduler with a Multilevel Feedback Queue (MLFQ) that separates I/O-bound processes (short bursts, high priority, small quantum) from CPU-bound processes (longer bursts, lower priority, larger quantum). Combined with asynchronous/non-blocking I/O so a process does not tie up a full context switch for every I/O call, this keeps the CPU supplied with runnable work and raises utilisation toward the 70–90% range typical of well-tuned systems.

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB
- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Answer: Q3

Memory issue

One instance of the program needs Code (5 MB) + Heap (150 MB) + Stack (25 threads $\times$ 1 MB = 25 MB) $\approx$ 180 MB. That alone is small next to 2 GB, so the fault rate is not caused by one process exceeding RAM — it is caused by the resident working set being spread thinly across 25 concurrently active threads, each touching different pages of the 150 MB heap. With this many threads competing, the frames actually allocated to the process's working set (per the working-set model) are smaller than the pages the threads are actively referencing at any instant, especially once several such processes/instances run together and the OS's frame allocation per process shrinks. The result is that pages get evicted just before they are re-referenced, so faults recur — this is thrashing driven by an oversized, poorly-partitioned heap combined with excessive per-thread stack overhead relative to the frames the process is actually given.

Two optimisations

- 1) Reduce thread count and stack overhead by using a bounded thread pool (e.g., 8–10 worker threads) instead of 25 raw threads, cutting stack memory and improving locality since fewer threads compete for the working set at once.
- 2) Optimise heap usage and paging: use memory pooling / arena allocation to shrink and localise the 150 MB heap footprint (better spatial locality reduces distinct pages touched per unit time), and increase the frames allocated to the process (or apply the working-set / page-fault-frequency algorithm) so the resident set matches actual reference locality instead of being starved.

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Answer: Q4

Root cause

The multilevel queue uses strict (absolute) priority scheduling between queues: Q3 (Batch, where P5 resides) is only serviced when both Q1 (Interactive) and Q2 (System) are completely empty. As long as interactive or system processes keep arriving — even in a steady low-intensity stream — Q1 and Q2 never both go empty at the same instant, so the scheduler never dispatches Q3 at all. This is classic indefinite postponement (starvation): P5 is ready and eligible, but a continuous supply of higher-priority arrivals means the no-preemption-of-priority rule between queues denies it CPU time indefinitely, even though the queue has no internal fairness problem of its own.

Optimisation for fairness

Introduce aging: track how long each Q3 process has waited and, after a threshold wait time, temporarily promote it to a higher queue (or grant it a guaranteed CPU slice) so its effective priority rises the longer it is starved, guaranteeing it eventually runs. A complementary fix is to convert the scheduler to Multilevel Feedback Queue with proportional/weighted time-sharing across queues — e.g., guarantee Q3 a minimum percentage of CPU time (weighted round robin across queues) rather than giving it access only when Q1 and Q2 are empty — which bounds P5's worst-case waiting time while still favouring interactive work.

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Answer: Q5

Why context switching is high

The time quantum (1 ms) is smaller than the average CPU burst (2 ms), so nearly every thread is preempted by the quantum expiry before it even finishes its burst — each burst now requires at least two dispatches instead of one, roughly doubling switches from scheduling alone. On top of that, very high I/O-wait frequency means threads additionally block mid-burst to perform I/O, triggering a second, independent source of context switches (voluntary switch on blocking, plus the switch back when the I/O completes and the thread is re-admitted to ready). The combination of an undersized quantum and I/O-driven blocking compounds into an extremely high switch rate, and since each switch carries fixed overhead (register/TLB/cache reload), the CPU spends a large fraction of every 2 ms burst doing switching work rather than useful computation.

Two optimisations

- 1) Increase the time quantum to be closer to (or slightly above) the average burst time (e.g., 2–4 ms) so most bursts complete within a single quantum, cutting quantum-driven preemptions substantially while still keeping response time reasonable.
- 2) Use an I/O-aware / multilevel feedback scheduling policy (or asynchronous non-blocking I/O) so threads that block frequently are handled with lower per-switch cost — e.g., lightweight user-level threading or I/O completion callbacks instead of a full kernel context switch for every I/O wait — reducing the number of expensive switches triggered purely by I/O waiting.

Q6. Semaphore Synchronization Debugging

A producer-consumer application occasionally hangs during execution. The semaphore values observed are:

```
mutex = 1
empty = 0
full = 10
Buffer size = 10
```

Producer Code:

```
wait(mutex)
wait(empty)
produce item
signal(full)
signal(mutex)
```

Consumer Code:

```
wait(mutex)
wait(full)
consume item
signal(empty)
signal(mutex)
```

Identify the synchronization bug causing the system to hang.

Explain which deadlock condition is being violated.

Suggest an optimization to ensure correct synchronization without changing the producer-consumer functionality. (10 Marks)

Answer: Q6

Synchronization bug

Both the producer and the consumer acquire the binary mutex before waiting on the counting semaphore (empty/full). If the buffer becomes full, the producer executes wait(mutex) successfully and then blocks on wait(empty) — while still holding mutex. The consumer, which needs mutex to consume an item and eventually signal(empty), can never acquire it because the producer is holding it while blocked. Symmetrically, if the buffer is empty, the consumer blocks on wait(full) while holding mutex, preventing the producer from ever entering to produce and signal(full). Either scenario freezes both processes permanently — the mutex is held across a blocking wait, so the very process that could unblock the waiter can never run.

Deadlock condition present

This is the hold-and-wait condition (a process holds mutex while waiting for another resource — empty/full — held/controlled by the other process), which in turn creates a circular wait between the two processes: producer waits for the consumer to free a slot while holding mutex, and the consumer needs mutex to free that slot.

Optimization

Reorder the semaphore operations so the counting semaphore is acquired before the mutex, without changing what the producer or consumer actually do: Producer — wait(empty); wait(mutex); produce item; signal(mutex); signal(full). Consumer — wait(full); wait(mutex); consume item; signal(mutex); signal(empty). Now a process only blocks on empty/full before it holds the mutex, so it never sits inside the critical section holding mutex while

waiting on another semaphore — the hold-and-wait condition is eliminated and the same producer-consumer behaviour is preserved.

Q7. Thrashing Detection and Optimization

A virtual memory system reports the following statistics:

- Number of processes: 50
- Average page fault rate: 250 faults/sec
- CPU Utilization: 18%
- Disk Utilization: 95%
- Free Frames Available: 2%

The operating system suspects thrashing.

Debug the root cause of the performance degradation.

Explain the relationship between page faults and CPU utilization.

Propose two operating system optimizations to reduce thrashing. (10 Marks)

Answer: Q7

Root cause

With 50 processes competing for frames but only 2% of frames free, the degree of multiprogramming has been pushed past the point where each process's working set fits in memory. Processes constantly fault, and while faulting they wait for the disk, which is why disk utilization is saturated (95%) while CPU utilization is very low (18%) — the CPU is idle waiting on page-ins rather than running instructions. This is textbook thrashing: the system is spending nearly all its time paging rather than executing.

Page faults vs CPU utilization relationship

As the degree of multiprogramming increases, CPU utilization initially rises (more ready processes fill idle CPU time), but beyond a critical point each process has too few frames to hold its working set, so the page-fault rate rises sharply; every fault forces the process to block for disk I/O, so the CPU sits idle waiting rather than computing. This drives CPU utilization down, which a naive scheduler may misread as 'CPU is idle, admit more processes' — admitting more processes shrinks each process's frame allocation further, raising the fault rate again. The result is the classic inverted relationship: past the thrashing point, more processes and more faults produce less CPU utilization, not more.

Two optimizations

- 1) Reduce the degree of multiprogramming by suspending or swapping out some processes (working-set based admission control) so the remaining processes' working sets fit within available frames, restoring locality and cutting the fault rate.
- 2) Use the working-set model or Page-Fault-Frequency (PFF) algorithm to dynamically allocate frames per process based on observed locality, only admitting a new process when enough free frames exist to satisfy its working set, preventing the system from re-entering the thrashing region.

Q8. File System Performance Debugging

A database server experiences slow file access times. System logs indicate:

- Average file size: 500 MB
- Disk seeks per request: 120
- Disk utilization: 92%
- Cache hit ratio: 15%

Identify the file system bottleneck affecting performance.

Explain why disk access latency is increasing.

Suggest two optimizations to improve file access efficiency. (10 Marks)

Answer: Q8

Bottleneck

The bottleneck is a combination of a very low buffer/page-cache hit ratio (15%) and severe file fragmentation: because only 15% of reads are served from cache, roughly 85% of every request must go to physical disk, and with 120 seeks needed per request the file's blocks are clearly scattered non-contiguously across the disk rather than laid out sequentially. Disk utilization is already at 92%, meaning the disk is nearly saturated servicing this seek-heavy, cache-miss-heavy workload, so requests queue up and access time balloons.

Why latency is increasing

Each of the 120 seeks per request adds mechanical seek time plus rotational latency before any data transfer even begins, and because the cache absorbs so few requests (15% hit ratio), almost every one of these expensive seek-laden reads actually reaches the disk instead of being answered instantly from memory. With disk utilization already at 92%, additional requests must wait in queue for the disk to become free, so queuing delay compounds the per-request seek overhead, driving overall access latency sharply upward.

Two optimizations

- 1) Increase and tune the buffer/page cache (larger cache size, better replacement policy such as LRU-K or read-ahead/prefetching for sequential database scans) to raise the hit ratio and reduce the fraction of requests that need to touch disk at all.
- 2) Reduce fragmentation and seeks through contiguous or extent-based file allocation (or periodic defragmentation/reorganization of the database files), and consider striping data across multiple disks (RAID) or moving hot files to SSD storage to cut per-request seek count and latency.

Q9. Thread Scheduling Debugging

A web server creates one thread per client request. During peak hours:

- Active Threads: 5,000
- CPU Utilization: 60%
- Memory Usage: 92%
- Average Response Time: 12 seconds

Scheduler logs show frequent thread creation and termination.

Identify the primary issue affecting server performance.

Explain how thread management contributes to the delay.

Recommend two optimizations to improve scalability and response time. (10 Marks)

Answer: Q9

Primary issue

The server uses a one-thread-per-request model, so at peak load it is running 5,000 concurrently live threads. This is a thread-management/resource-exhaustion problem, not primarily a CPU-capacity problem: memory usage is at 92% (each thread's stack and kernel bookkeeping consumes memory) while CPU utilization is only 60%, showing the CPU is not the limiting resource — the system is being strangled by the overhead of managing thousands of threads rather than by lack of raw compute.

How thread management causes the delay

Every incoming request pays the cost of thread creation (kernel object allocation, stack allocation) and later thread termination/cleanup, and the scheduler must context-switch among 5,000 runnable/blocked threads, so an increasing share of CPU time goes to scheduling and switching overhead instead of request processing. Memory pressure from thousands of per-thread stacks pushes usage to 92%, risking swapping, which would add disk I/O latency into the response path. The combined effect — creation/termination overhead, high context-switch cost among 5,000 threads, and memory pressure — inflates average response time to 12 seconds even though the CPU itself is not fully utilized.

Two optimizations

- 1) Replace one-thread-per-request with a bounded thread pool (a fixed or dynamically-bounded set of worker threads that pick up requests from a queue), eliminating per-request thread creation/termination overhead and capping memory usage.
- 2) Move to an event-driven / asynchronous, non-blocking I/O architecture (e.g., an event loop or reactor pattern with a small number of worker threads handling many connections), which scales to thousands of concurrent clients without one OS thread per connection, reducing both scheduling overhead and memory footprint.

Q10. Disk Scheduling Optimization Problem

A disk scheduling log shows the following request queue:

98, 183, 37, 122, 14, 124, 65, 67

Current Head Position: 53

The system uses FCFS scheduling and reports:

- Average seek time: Very High
- Throughput: Low

User complaints regarding slow response. Debug why FCFS is causing poor performance in this scenario.

Compare FCFS with one suitable disk scheduling algorithm for this workload.

Propose an optimization to improve seek time and overall throughput. (10 Marks)

Answer: Q10

Why FCFS performs poorly

FCFS services requests strictly in arrival order — 53→98→183→37→122→14→124→65→67 — regardless of their physical position on the disk. This forces the read/write head to swing back and forth across the cylinder range repeatedly instead of moving in a consistent sweep. Computing the head movement:
 $|53-98| + |98-183| + |183-37| + |37-122| + |122-14| + |14-124| + |124-65| + |65-67|$
 $= 45+85+146+85+108+110+59+2 = 640$ cylinders of total seek movement to service just 8 requests. This large, oscillating travel distance is exactly why average seek time is very high and throughput is low.

Comparison with SCAN (elevator algorithm)

SCAN sorts pending requests and services them in one sweep direction before reversing, minimizing back-and-forth travel. Starting at head 53 and sweeping upward first: 53→65→67→98→122→124→183 (130 cylinders), then reversing to service the remaining lower requests 183→37→14 (169 cylinders), giving a total of 130+169 = 299 cylinders — well under half of FCFS's 640. SCAN (and its variants LOOK/C-SCAN) consistently outperform FCFS whenever requests are scattered across a wide cylinder range, because it eliminates redundant reversals and services nearby requests together.

Optimization

Replace FCFS with SCAN/LOOK (or C-SCAN for more uniform wait times) at the disk-scheduling layer so requests are reordered by cylinder position rather than arrival time, cutting total head movement by roughly 53% in this scenario. This directly reduces average seek time and increases throughput; combining it with request queue merging/sorting and read-ahead caching for the database's sequential access patterns would further reduce response-time complaints from users.

Rubrics for Calculation

Criteria (Weightage)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification 20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging 25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization 20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis 20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality 15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation